

What we need from SwyChr to finish the integration.

Eight items, in the order that unblocks the most. Three of them were raised at the technical meeting on 29 September and are still open; we have repeated them here so everything sits in one place.

FROM	DATE	MARKET	REPLACES
Groundwork by Jalla	7 October 2026	Cameroon	Brief of 29 Sept 2026

IN ONE PARAGRAPH

Groundwork is a construction platform. A client abroad funds a build at home; we track the work stage by stage, and when a stage is verified, the contractor on site is paid. **Groundwork never holds money** — SwyChr does. We hold the record of what moved, for whom, against which stage. We have built everything on our side up to the point of sending a payout. The items below are what we cannot build without you.

WHAT ALREADY WORKS, SO THIS IS NOT A LIST OF COMPLAINTS

Your pay-in flow works. We receive the `payment_link_status_updated` webhook and record funds arriving. Our scheduled job already polls `/api/payout/transaction_status` every five minutes and is ready to use. `/api/payout/payout_methods` returns mobile money operators per country, which we use. The gap is narrow and specific: **it is the payout half.**

The eight things we need

Each item says what we are asking for, why it matters in plain terms, and what would close it. Items marked *raised 29 Sept* are repeats.

1 A test environment

NEW

WHAT WE NEED	Sandbox credentials where we can create a payout and see it succeed and fail, without real money moving.
WHY IT MATTERS	We are being asked to write code that sends people's money. We will not write that code and test it for the first time on a live transaction to a real contractor. If the only way to test is with real funds, the integration cannot be completed responsibly.
CLOSED WHEN	We have sandbox credentials and can create one successful and one failed test payout.

2 A payout webhook — or confirmation there is none

RAISED 29 SEPT · OPEN

WHAT WE NEED	A webhook that fires when a payout changes state, in the same way <code>payment_link_status_updated</code> fires for pay-ins. If no such webhook exists and none is planned, please say so plainly.
WHY IT MATTERS	Today we find out what happened to a payout only by asking you repeatedly about an ID we already hold. That works, but it means a contractor can be paid minutes before we know it, and a failure can sit unnoticed until our next check. A plain "no webhook, use polling" is a perfectly good answer — we have already built the poller. What costs us is not knowing which it is.
CLOSED WHEN	We have either the webhook specification, or written confirmation that polling <code>transaction_status</code> is the supported method.

3 A way to list transactions for a period

RAISED 29 SEPT · OPEN

WHAT WE NEED	An endpoint that returns all transactions on our account between two dates — not one transaction by ID, but the list.
WHY IT MATTERS	Every endpoint you publish answers "what happened to this ID?". That only works for transactions we created. We cannot discover one we did not create, which means we cannot check at month end that our records and yours agree. If a payout exists on your side that we have no record of, there is currently no way for either of us to find out.
CLOSED WHEN	We can request a date range and receive every transaction in it.

4 Our reference carried on transactions we did not create

RAISED 29 SEPT · OPEN

WHAT WE NEED	A way to attach our own reference to a transaction a <i>client</i> starts or authorises on your platform, and to read it back.
WHY IT MATTERS	When we create a transaction ourselves we can pass our own <code>transaction_id</code> , and that works well. But when the client confirms a payment on your side, we never learn the ID at all — so we cannot tie that money to a project, a stage, or a contractor. The money moves correctly and our record of it is blank.
CLOSED WHEN	A client-initiated transaction can carry a Groundwork reference that we can read back.

5 Your bank codes for Cameroon

NEW

WHAT WE NEED	The list of banks you can pay out to in Cameroon, with the exact value you expect in the <code>bank_code</code> field of <code>create_transaction</code> . We already use <code>/api/payout/payout_methods</code> for mobile money operators — please confirm whether that same endpoint returns banks, or whether there is a separate list.
WHY IT MATTERS	We store our own internal identifier for each bank, deliberately, because we do not know your codes. We will not guess them. A wrong bank code does not fail loudly — it sends a contractor's money somewhere, and nobody finds out for days.
CLOSED WHEN	We have the list, and know which endpoint returns it.

6 The exact payout request for Cameroon

NEW

WHAT WE NEED	For <code>/api/payout/create_transaction</code> , a worked example for Cameroon — one for mobile money and one for bank transfer — showing every required field and its exact format. In particular: the expected format of <code>mobile_no</code> , which fields are required for a bank payout, and whether sending a duplicate <code>transaction_id</code> is safely rejected or creates a second payment.
WHY IT MATTERS	That last point is the one that worries us most. If our server sends a payout request and the connection drops before we hear back, we must know whether retrying is safe. If a repeat can pay a contractor twice, we have to build differently.
CLOSED WHEN	We have two working examples and a clear answer on duplicate handling.

7 The full list of statuses you can return

NEW

WHAT WE NEED	Every value <code>status</code> can take on a payout and on a pay-in — including every failure, expiry, decline and rejection — and which of them are final.
WHY IT MATTERS	We show a client and a contractor what happened to their money. If we meet a status we have never seen, the honest thing is to show nothing, which looks broken. We also need to know which statuses are final, so we stop asking about a transaction that is finished.
CLOSED WHEN	We have the list, marked final or not final.

8 Confirmation of our account and a named contact

NEW

WHAT WE NEED	Confirmation that our API key, webhook secret and callback URL are correctly registered on your side and which environment they belong to — plus one named person we can reach when a payment behaves unexpectedly.
WHY IT MATTERS	We have the credentials configured but have never seen a payout complete, so we cannot tell a configuration problem from an integration problem. And once real money is moving, "who do we call" cannot be an open question.
CLOSED WHEN	Our setup is confirmed and we have a contact and a response expectation.

Where this leaves us

This is the chain we need to complete. The first two steps work today. The rest is what the eight items above unblock.

#	STEP	TODAY	NEEDS
1	Client funds the project	Works	—
2	We record the money as received	Works	—
3	Stage is verified; a person authorises the release	Works	—
4	Our server asks SwyChr to pay the contractor	Not built	Items 1, 5, 6
5	We store your reference for that payout	Blocked by 4	Item 6
6	We learn the outcome	Poller ready	Items 2, 7
7	Month end: our records and yours agree	Not possible	Items 3, 4

WHAT WOULD UNBLOCK US FASTEST

If you can only act on two things this week, make them **item 1** (a test environment) and **item 6** (the exact payout request, including whether a retry can pay twice). With those we can build and safely test the payout call. Items 5 and 7 can follow, and mobile money can go live before bank transfers if your bank list takes longer.

We are ready to build as soon as these land. Everything on our side up to the payout call is finished and tested, and our status poller is already running on a five-minute schedule waiting for its first real transaction.

Groundwork by Jalla

Reply to this document point by point if that is easiest — numbered answers against numbered asks.